

伪随机数和序列密码

清华大学计算机系
于红波

2026-03-31



提纲

- 伪随机数
- 序列密码基本概念
- 线性反馈移位寄存器LFSR
- 序列和周期
- 基于LFSR的伪随机序列生成器
- 实用序列密码算法



使用随机数的密码技术

□生成密钥

- 用于对称密码和消息认证码

□生成密钥对

- 用于公钥密码和数字签名

□生成初始化向量（IV）

- 用于分组密码的CBC、CFB和OFB模式

□生成 nonce

- 用于防御重放攻击以及分组密码的CTR模式等

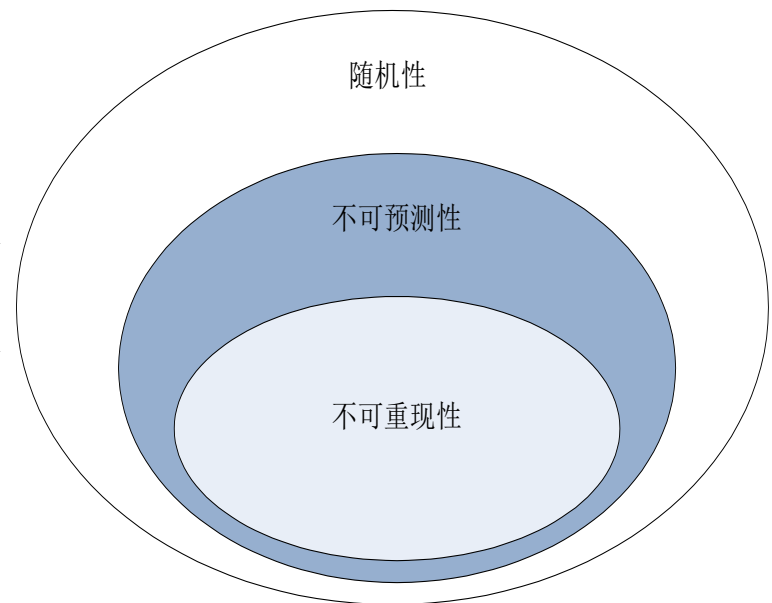
□生成盐

- 用于基于口令的密码（PBE）等



一、随机数的性质

- **随机性**：不存在统计学偏差，是完全杂乱的数列
 - **不可预测性**：不能从过去的数列推测出下一个出现的数
 - **不可重现性**：除非数列的本身保存下来，否则不能重现相同的数列
-
- 三个性质中，越往下就越严格。
 - 具备随机性，不代表一定具备不可预测性。
 - 具备不可预测性的随机数，一定具备随机性。
 - 具备不可重现性的随机数，也一定具备随机性和不可预测性。





随机数的分类

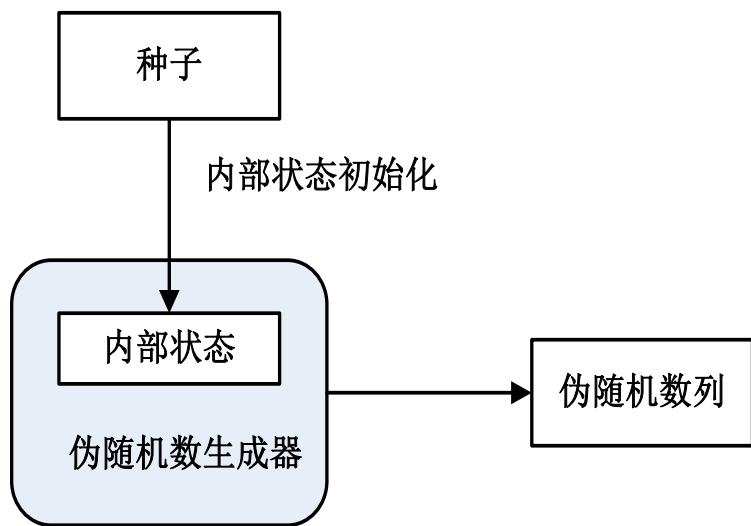
	随机性	不可预测性	不可重现性	
弱伪随机数	√	X	X	只具备随机性
强伪随机数	√	√	X	具备不可预测性
真随机数	√	√	√	具备不可重现性

● 密码技术中所使用的随机数，仅仅具备随机性是不够的，至少还需要具备不可预测性。

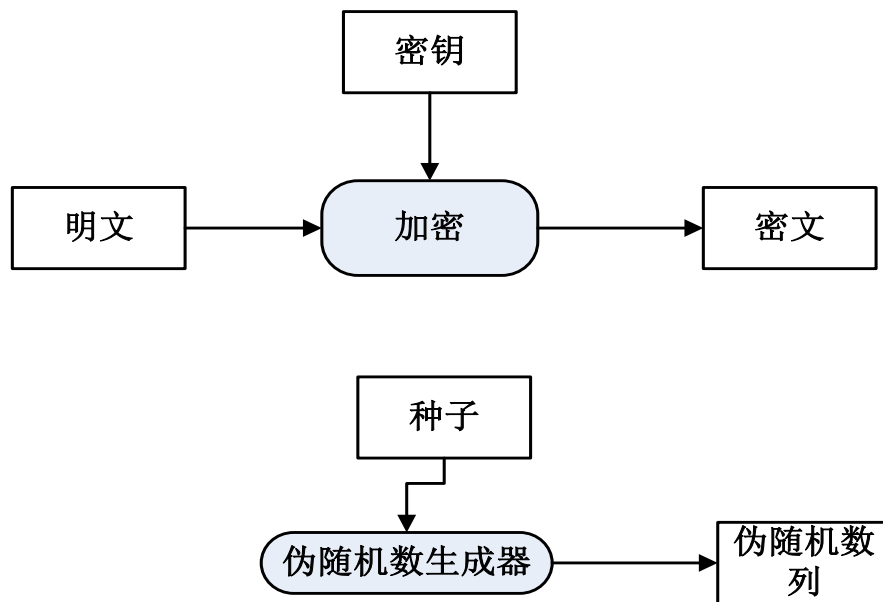


伪随机数生成器

- ❑ 伪随机数生成器具有“内部状态”，并根据外部输入的“种子”来生成伪随机数序列。
- ❑ 伪随机数的“种子”是一串随机的比特序列，伪随机数生成器是公开的，但种子需要自己保密，类似于密码算法中的密钥。



伪随机数生成器的结构



密码的密钥与伪随机数的种子



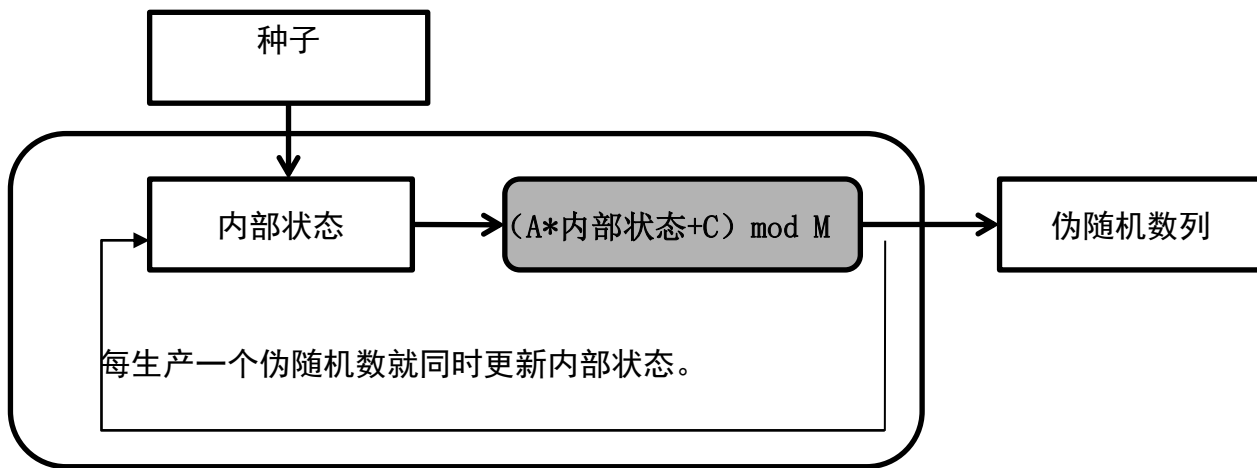
具体的伪随机数生成器

- 线性同余法
- 单向散列函数法
- 密码法



线性同余法

线性同余法是一种使用很广泛的随机数生成器算法，很多伪随机数生成器的库函数（library function）都是采用线性同余法编写的。例如C语言的库函数rand，以及Java的java.util.Random类等。线性同余法的程序代码



用线性同余法 实现伪随机数生成器

简而言之，线性同余法就是将当前的伪随机数乘以A再加上C，然后将除以M得到的余数作为下一个伪随机数。



线性同余法

□ 线性同余法不具备不可预测性，不可以将线性同余法用于密码技术。

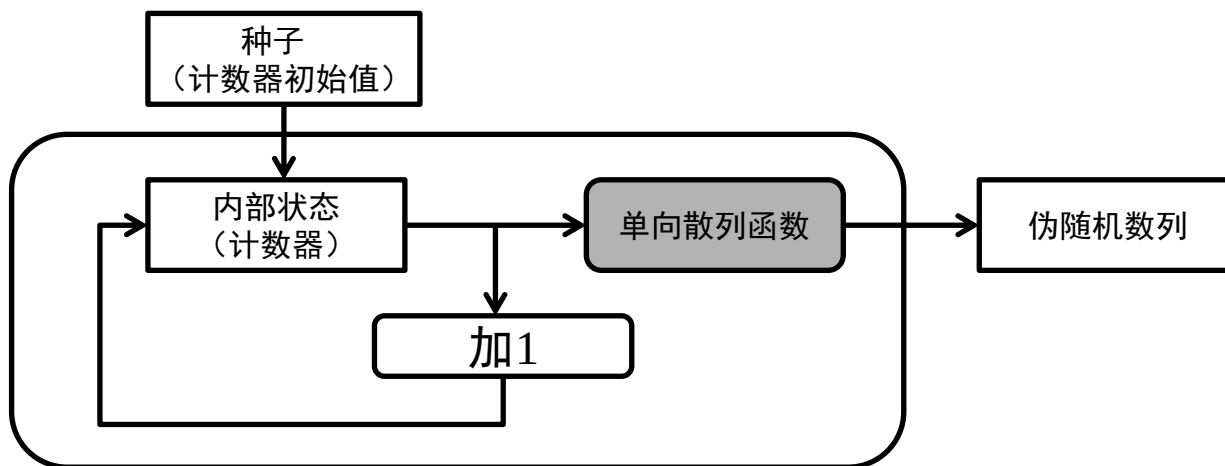
$$\text{伪随机数} = (A * \text{种子} + C) \bmod M;$$

□ A, C, M 都是常量。这时攻击者只要得到所生成的伪随机数中的任意一个，就可以预测出下一个伪随机数。



单向散列函数

- 使用单向散列函数（如SHA-1）可以编写出能够生成具备不可预测性的伪随机数列的伪随机数生成器。
- 单向散列函数的单向性是支撑伪随机数生成器不可预测性的基础。

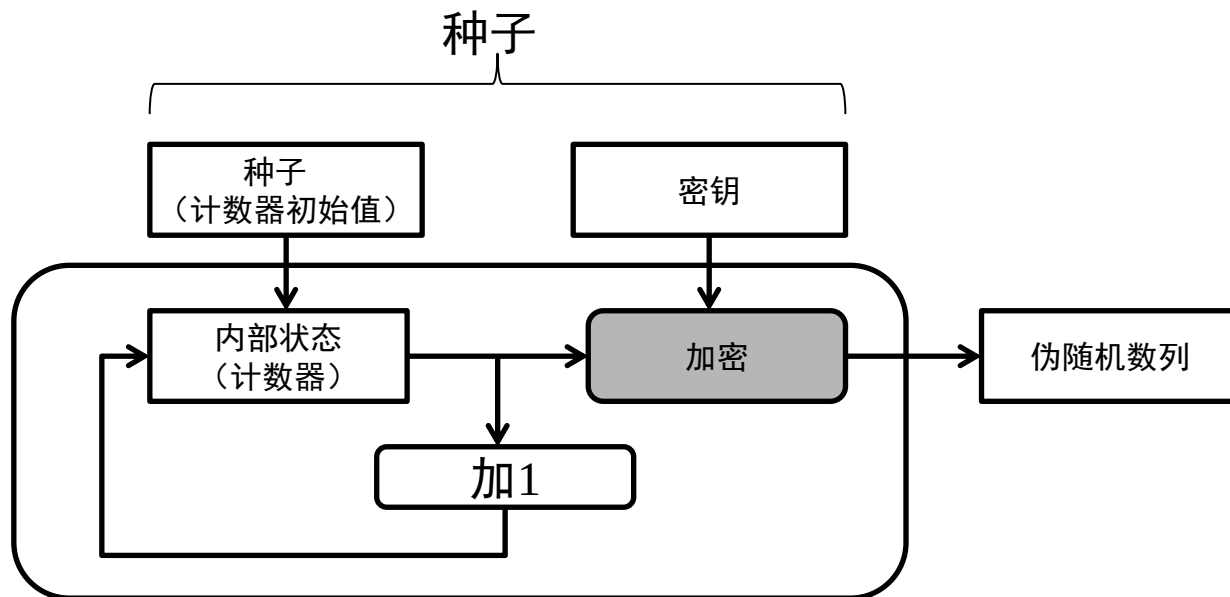


用单向散列函数实现伪随机数生成器



密码法随机数生成器

- ❑ 使用密码可以用来构造伪随机数生成器。既可以使用AES等对称密码，也可以使用RSA等公钥密码。
- ❑ 密码的机密性是支撑伪随机数生成器不可预测性的基础。



用密码实现伪随机数生成器



(二) 序列密码的基本概念

□ 现代密码体制分为：

加密算法：

对称加密--分组密码和流密码（序列密码）

非对称加密--公钥密码。

认证算法：Hash、消息认证码、数字签名

□ 序列密码

针对明文消息的单个字符（二进制位）进行加密解密变换。

算法简单、速度快、错误传播少。



(二) 序列密码的基本概念

序列密码的基本思想：是利用密钥 k 产生一个密钥流 $k=k_1k_2\dots$ ，并使用如下规则对明文串 $m=m_1m_2m_3\dots$ 加密得到密文 $c=c_1c_2c_3\dots$ ：

$$c=c_1c_2c_3\dots=E(m_1,k_1)E(m_2,k_2)E(m_3,k_3)\dots$$

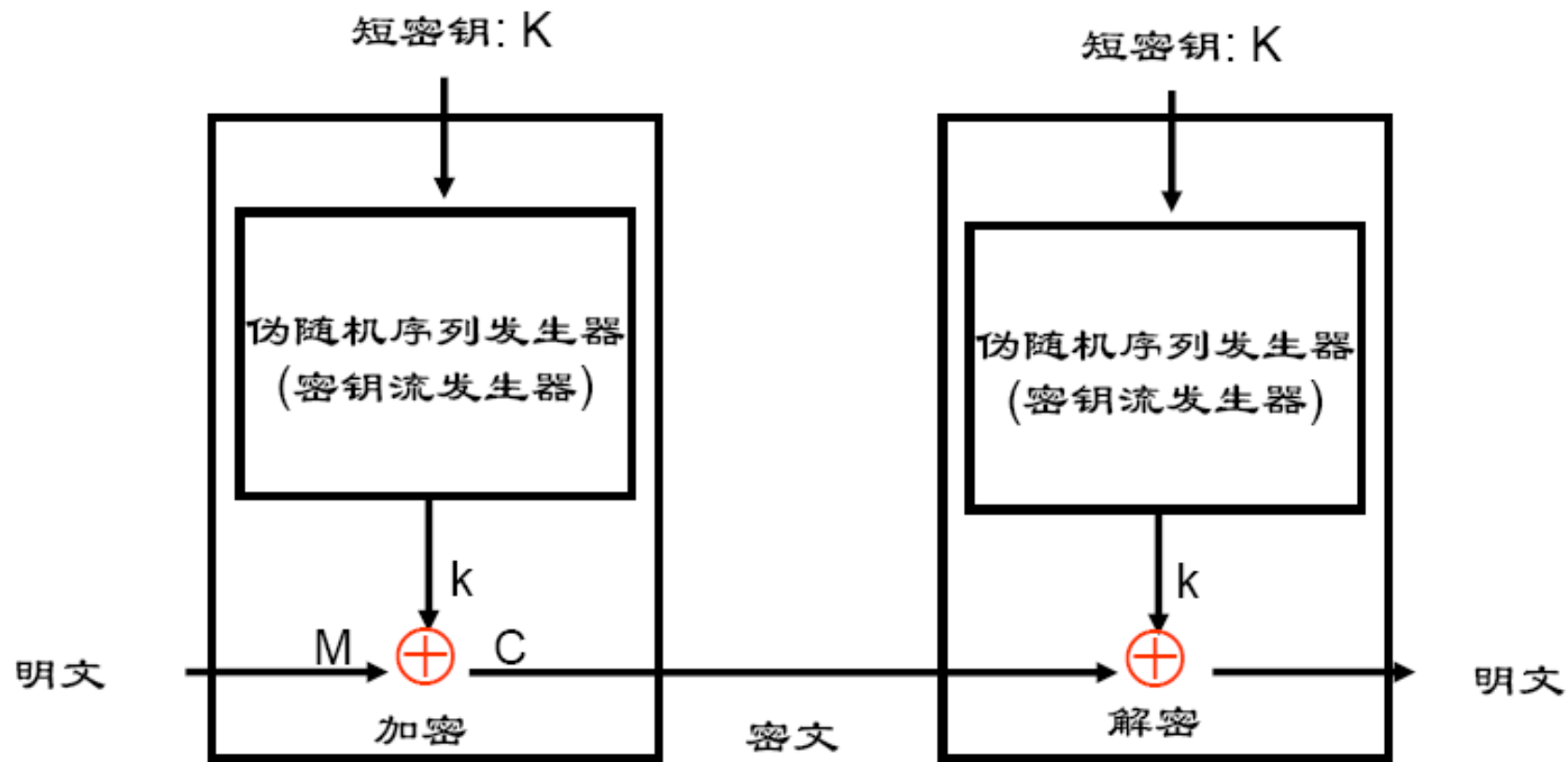


图 序列密码系统模型



(二) 序列密码的基本概念

注意：

- **密钥流是随机的，但要能重复生成。使用具有确定性的密钥流生成器。**
- **生成器的输入是一个短的易记住的密钥，称为初始密钥或种子密钥。**



序列密码的结构

分类：同步序列密码和自同步序列密码。

一、同步序列密码

同步序列密码：密钥序列产生与明文和密文无关。

在通信的过程中，通信的双方必须保持精确的同步，收方才能正确解密，如果失去同步将不能正确解密。

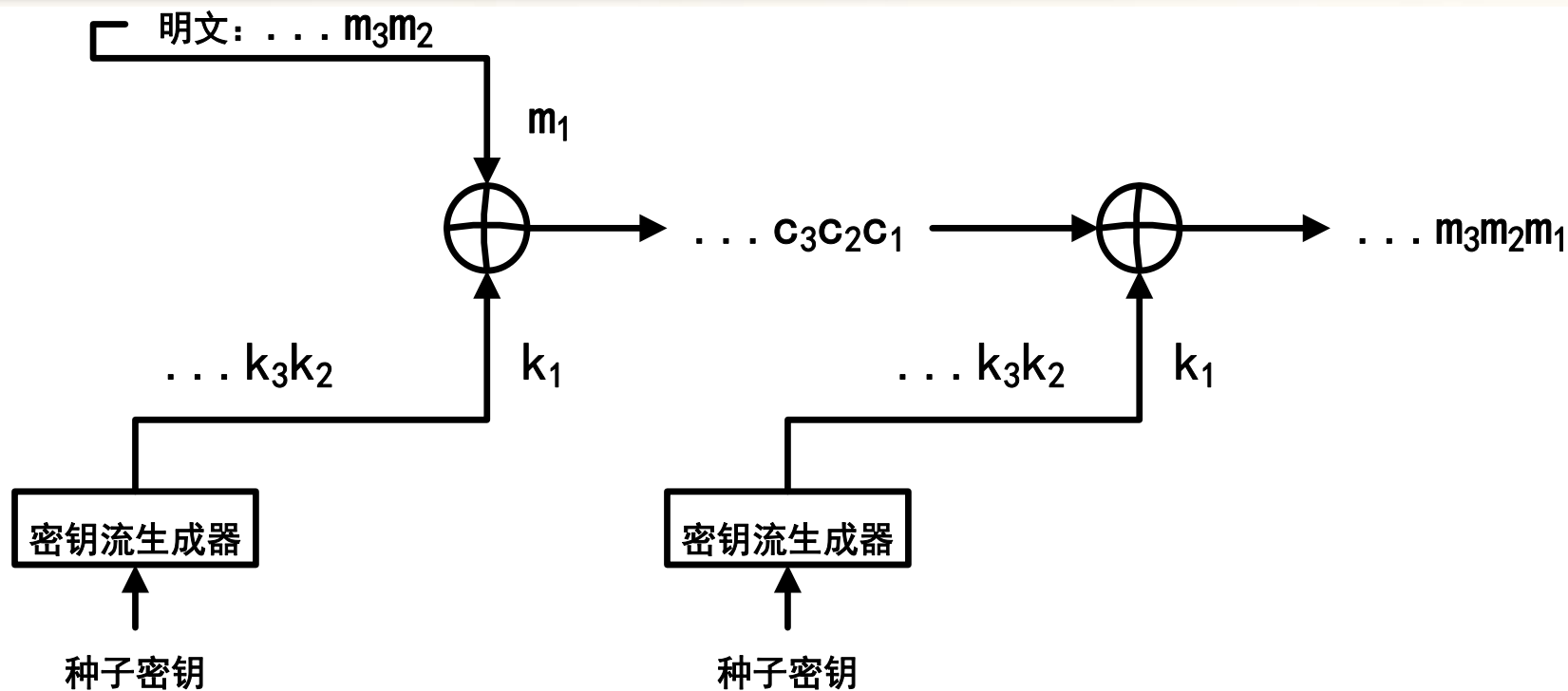


图 同步序列密码结构

加密算法: $c_i = m_i \oplus k_i$

解密算法: $m_i = c_i \oplus k_i$



问题：如果密文 c_2 丢失，影响多少个明文不能正确解密？

$$\begin{array}{ll} m = m_1, m_2, m_3, \dots, m_n & \text{明文} \\ \oplus k = k_1, k_2, k_3, \dots, k_{n-1}, k_n & \text{密钥} \\ \hline c = c_1, c_2, \dots, c_{n-1} & \text{密文} \end{array}$$

问题：如果密文 c_2 错误，那么影响多少个明文不能正确解密？



同步序列密码

□优点:

- 容易检测出是否有插入、删除等主动攻击。
- 如果密文中只有某个字符产生了错误（不是插入或删除），只影响此字符的解密，不影响其他字符，**即无错误传播。**



自同步序列密码

- ❑ 密钥序列产生是密钥以及固定位数的以前密文位的函数。
- ❑ 设密钥序列产生器具有 n 位存储，则加密时一位密文错误将影响后面连续 n 个密文错误。在此之后将恢复正确。解密时一位密文错误也将影响后面连续 n 个密文错误。在此之后将恢复正确。
- ❑ 加密解密会造成错误传播。在错误之后恢复正确。

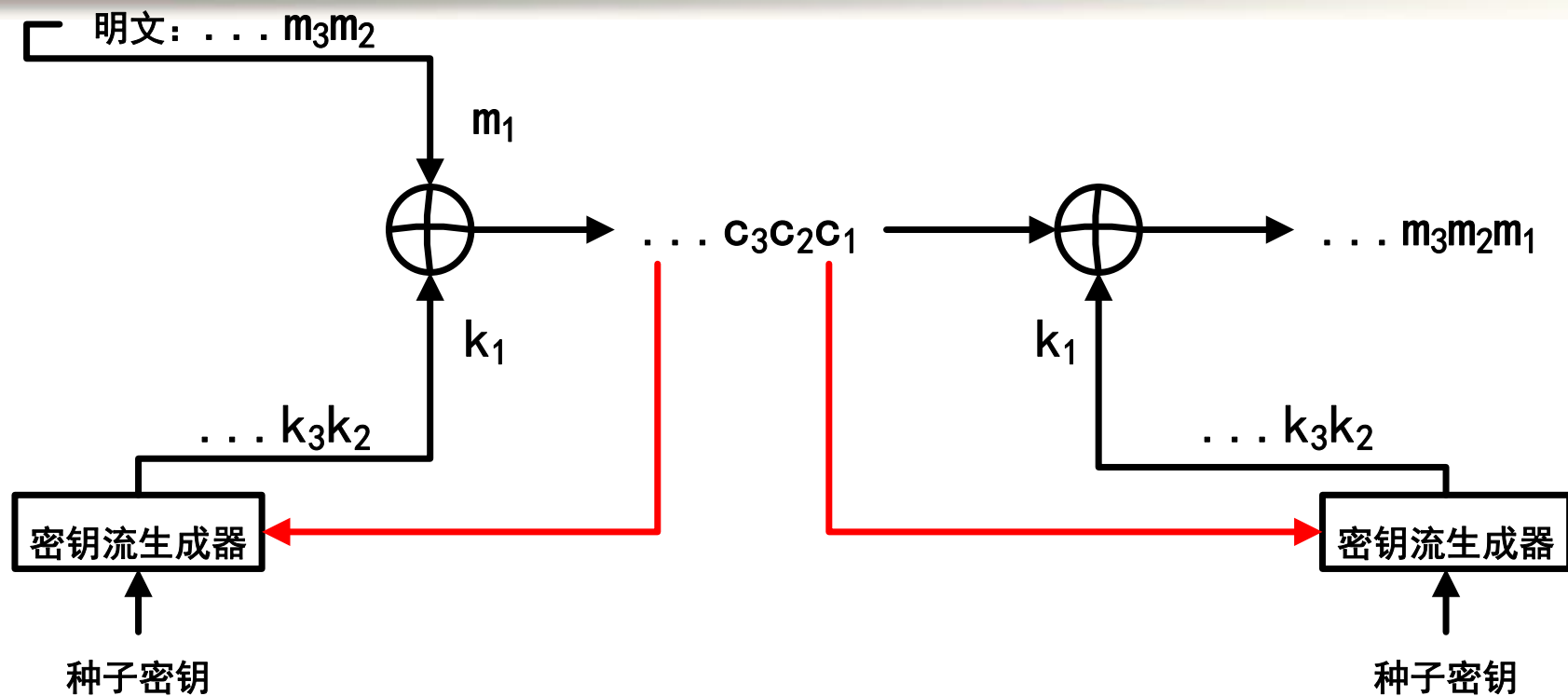


图 自同步流密码结构

加密算法: $c_i = m_i \oplus k_i$ 解密算法: $m_i = c_i \oplus k_i$
其中: $k_i = g(f_i(c_{i-n+1}, \dots, c_{i-1}, c_i), k)$



(三) 线性反馈移位寄存器

□ 线性反馈移位寄存器LFSR (linear feedback shift register) 是序列密码产生密钥流的一个主要组成部分。一个 n 级反馈移位寄存器由存储器与一个反馈函数 $f(a_1, a_2, \dots, a_n)$ 组成。

□ 特点：移入端移入一位，移出端移出一位。

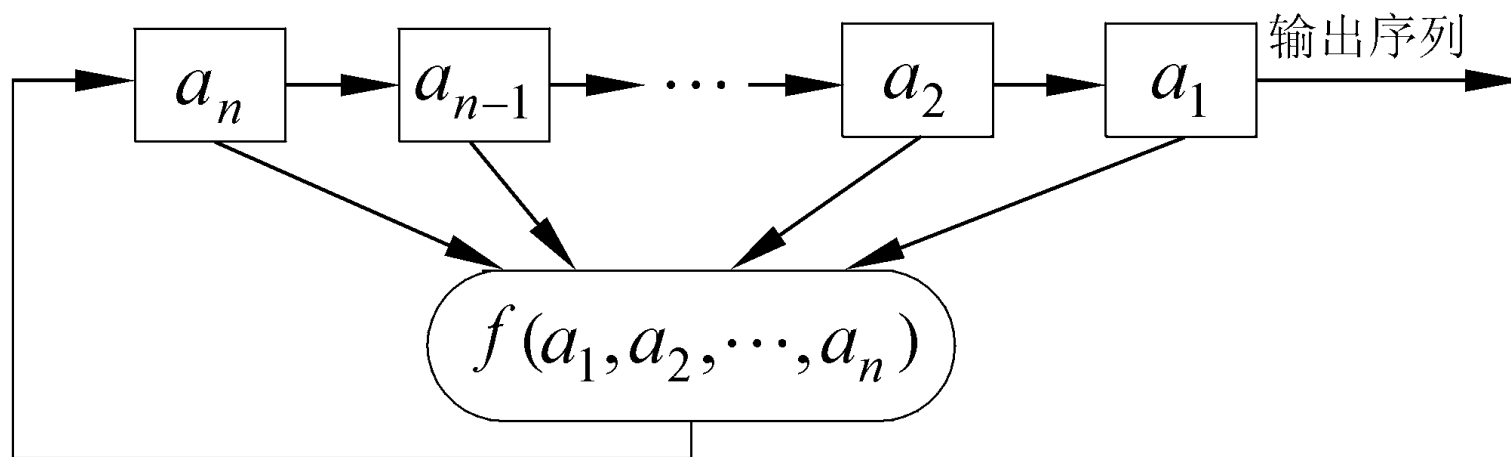
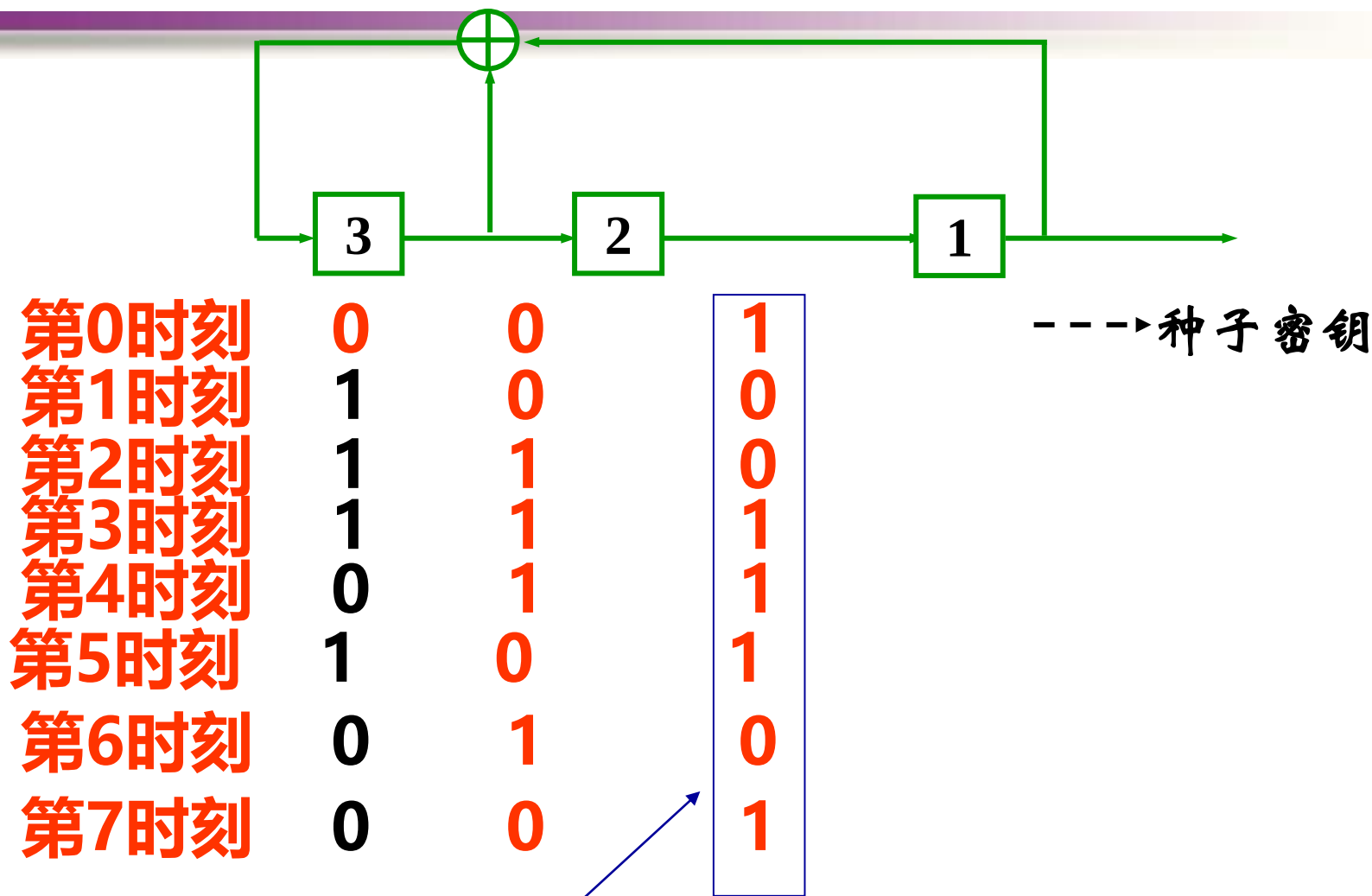


图 n 级反馈移位寄存器



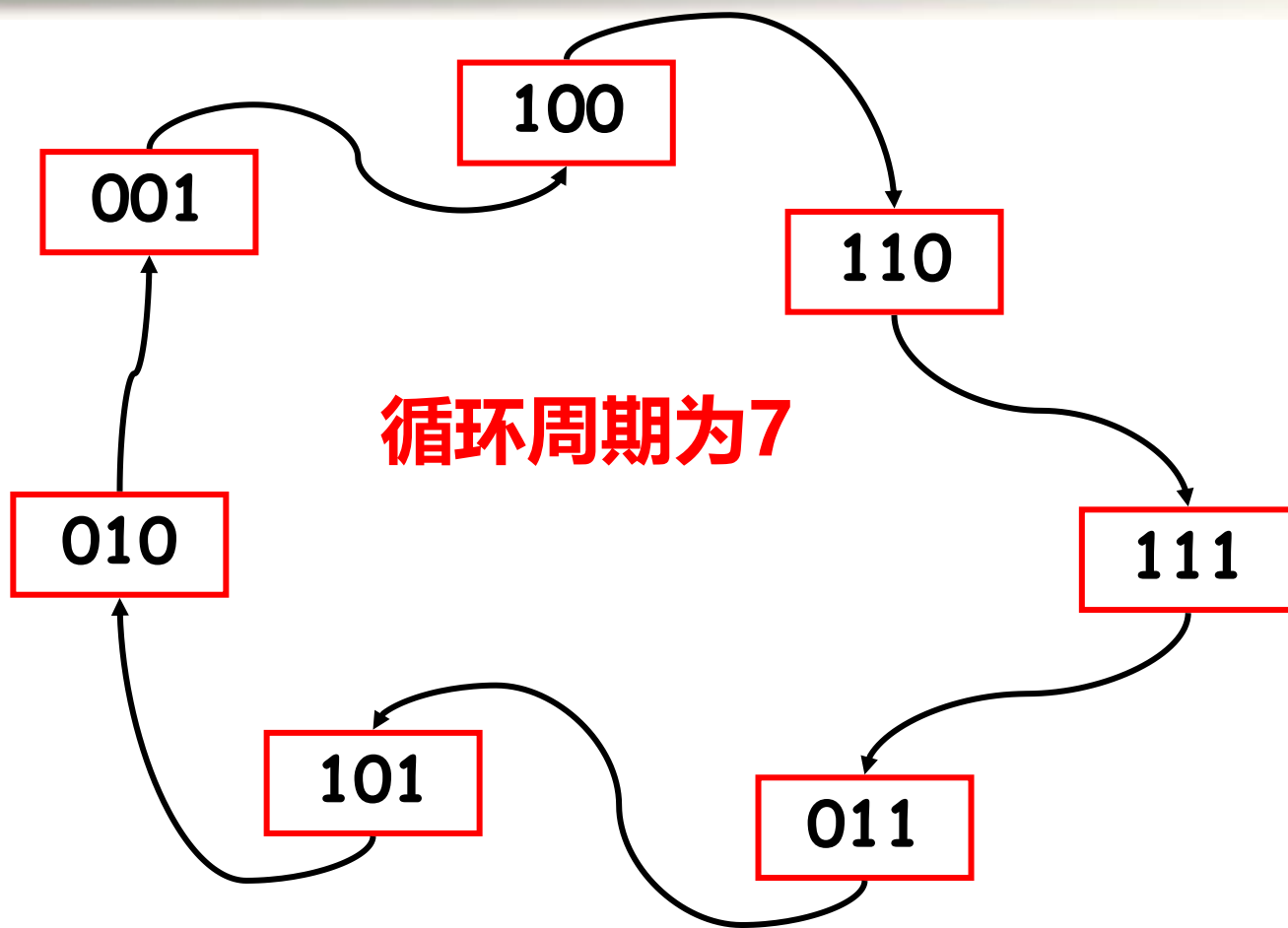
例1：3级LFSR，反馈函数 $f=a_3 \oplus a_1$



产生序列为：1001110.....



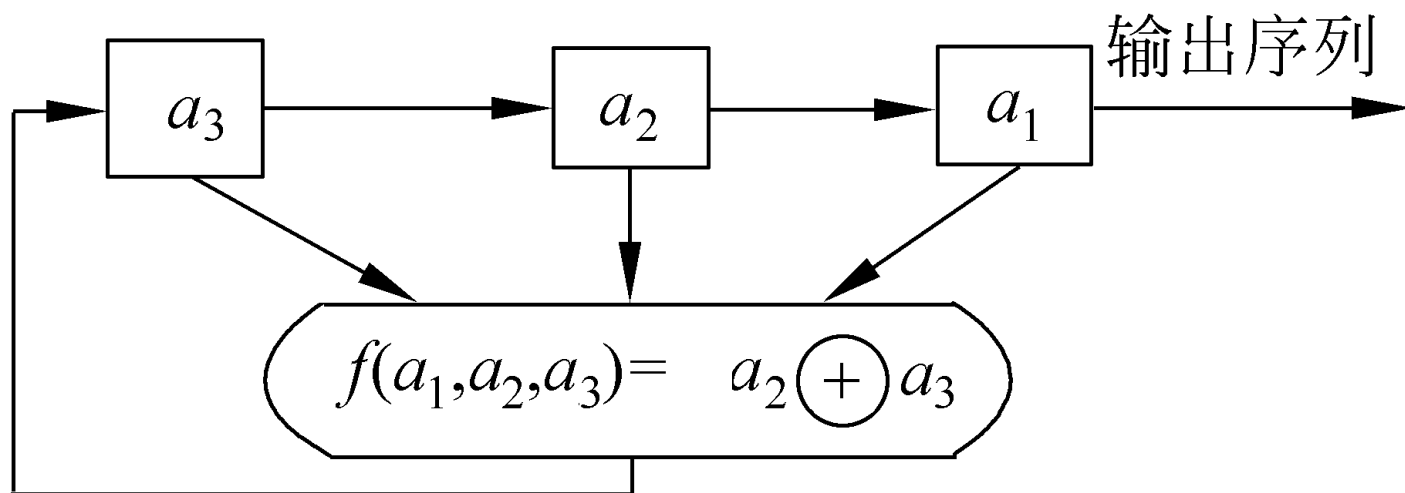
例1：3级LFSR，反馈函数 $f=a_3\oplus a_1$

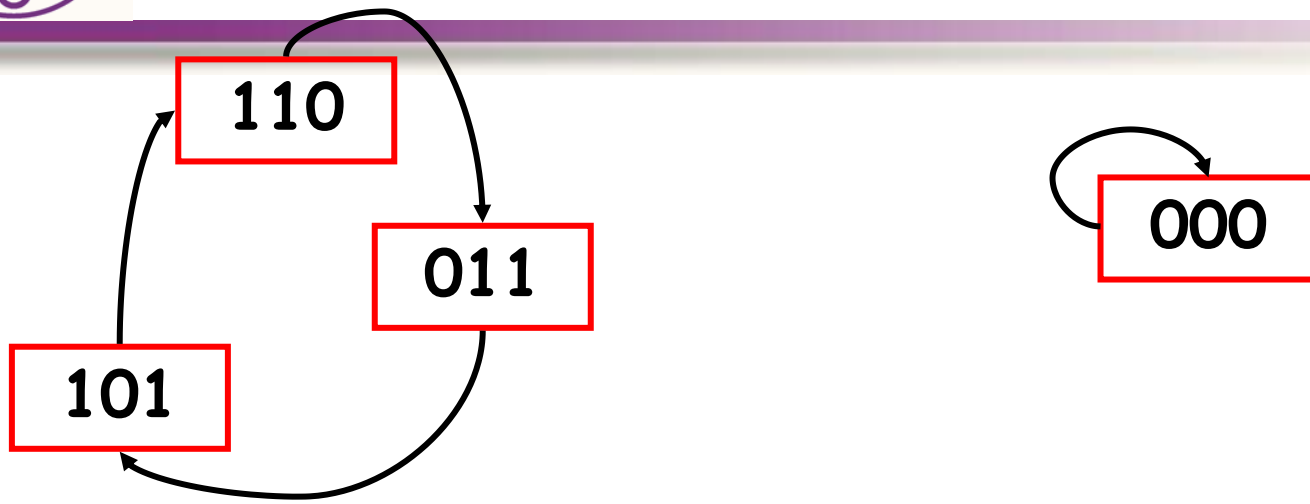


3级LFSR状态转移图



例2： 下图是一个3级反馈移位寄存器， 其初始状态为 $(a_1, a_2, a_3)=(1,0,1)$ 或 $(0,0,0)$ ， 输出序列是什么？





(1)输出序列为101..., 周期为3。

(2)输出序列为0..., 周期为1。

注意: n 级寄存器产生序列的周期 T 最大为 $2^n - 1$



特征多项式

设 n 级线性移位寄存器的输出序列 $\{a_i\}$ 满足递推关系

$$a_{n+k} = c_1 a_{n+k-1} \oplus c_2 a_{n+k-2} \oplus \dots \oplus c_n a_k$$

其中 $c_i=0$ 或 1 。

引入延迟算法 $D, D(a_i)=a_{i-1}$, 上式为

$$0 = a_{n+k} \oplus c_1 D a_{n+k} \oplus c_2 D^2 a_{n+k} \dots \oplus c_n D^n a_{n+k}$$

$$\text{即 } (1 \oplus c_1 D \oplus c_2 D^2 \oplus \dots \oplus c_n D^n) a_{n+k} = 0$$

这种递推关系可用一个一元高次多项式表示,

$$P(x) = 1 + c_1 x + \dots + c_{n-1} x^{n-1} + c_n x^n$$

称这个多项式为LFSR的**特征多项式**。



示例

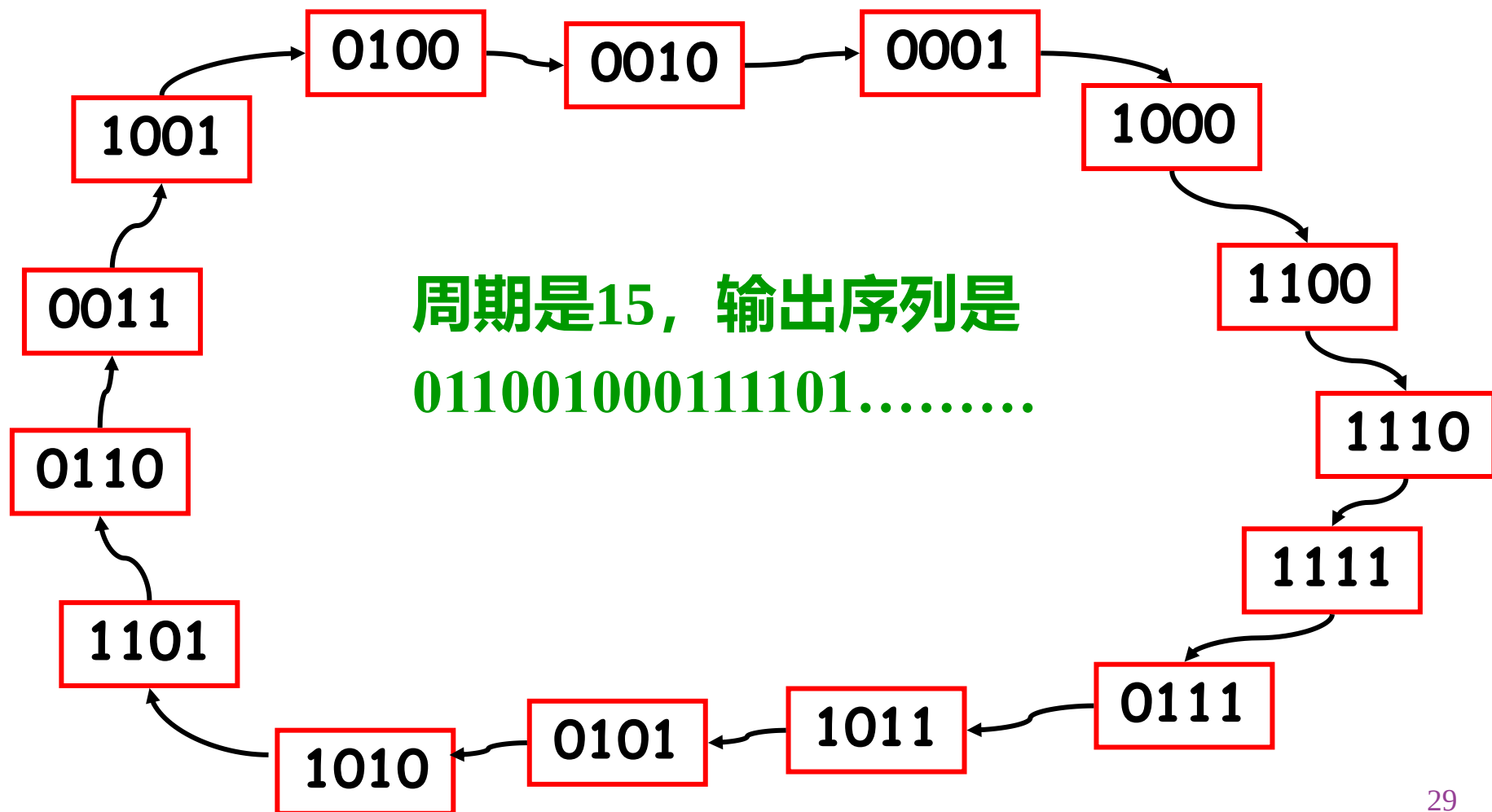
□ eg: 已知一个4级线性反馈移位寄存器如下所示, 求出输出序列是什么? 周期是什么?

若特征多项式为 $p(x)=1+x+x^4$ 。初始状态为
 $(a_1, a_2, a_3, a_4) = (0, 1, 1, 0)$

若特征多项式为 $p(x)=1+x+x^2+x^3+x^4$ 。若初始状态为 $(a_1, a_2, a_3, a_4) = (1, 1, 1, 1)$ 。

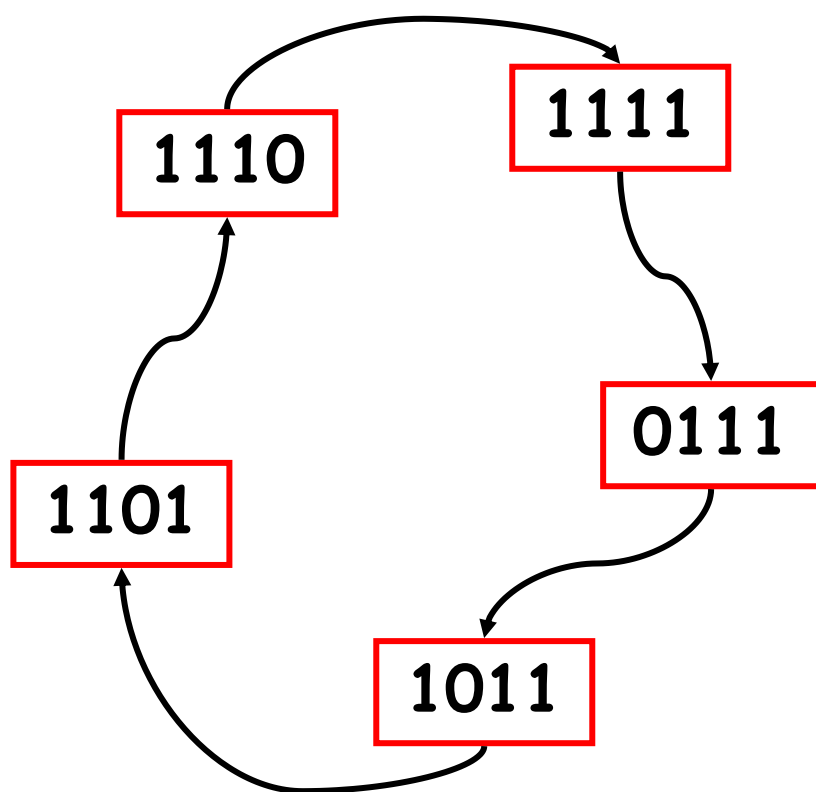


解：（1）由特征多项式 $p(x)=1+x+x^4$ 推得反馈函数 $f=a_1 \oplus a_4$ 。





(2) 由特征多项式 $p(x)=1+x+x^2+x^3+x^4$
推得反馈函数 $f=a_1 \oplus a_2 \oplus a_3 \oplus a_4$ 。



所以输出序列是
11110...,周期是5



（四）、序列和周期

对于序列 $\underline{a} = a_0 a_1 a_2 \dots a_i \dots$ ，若存在整数 p 使得对任意正整数 k 有 $a_k = a_{k+p}$ 成立，称满足该式的最小正整数 p 为**序列的周期**。

r 级线性反馈移存器的最长周期： $2^r - 1$ ，能达到最长周期的线性移存器序列称为 **m 序列**。

在密码学中，我们希望参与变换的序列周期越长越好，因此对线性反馈移存器我们更感兴趣的是能达到最长周期的序列，即 **m 序列**。



□ Fact:

1、一个LFSR $\langle f(x), l \rangle$ 是周期序列，当且仅当它的反馈多项式 $f(x)$ 的次数为 l 。

若 $f(x)$ 的次数低于 l ，则称LFSR $\langle f(x), l \rangle$ 是退化的 (singular)，由它输出的序列是终归周期的 (ultimately periodic)。

2、 $\langle f(x), l \rangle$ 生成序列是一个 m -序列当且仅当 $f(x) \in \mathbb{Z}_2[x]$ 是一个次数为 l 的本原多项式

不可约多项式： 设 $f(x) \in \mathbb{F}_2[x]$ 是一个次数大于1的多项式，如果 $f(x)$ 不能表成 $\mathbb{F}_2[x]$ 中的两个次数不小于1的多项式的乘积，我们就说 $f(x)$ 在 $\mathbb{F}_2[x]$ 上是不可约的。



本原多项式：一个次数为 n 的不可约多项式 $f(x) \in F_2[x]$ 称为一个本原多项式，如果 x 是乘群 $F_{2^n}^*$ 的生成元，其中 $F_{2^n}^*$ 是有限域 $F_{2^n} = F_2[x]/(f(x))$ 中的乘群。

Fact: 1. $f(x) \in F_2[x]$ 是一个次数为 n 的本原多项式，当且仅当 $f(x) \mid x^{2^n-1} - 1$ ，并且 $f(x)$ 不整除 $x^k - 1$ for $0 \leq k < 2^n - 1$

2. 当 $n \geq 1$, 共有 $\phi(2^n - 1)/n$ 个次数为 n 的本原多项式。当 $2^n - 1$ 为素数时， F_2 上每个 n 次不可约多项式都是本原多项式



随机性的描述

1. Golomb随机性假设

为了度量周期序列的随机性，**Golomb**提出了下列**三条标准**：

- (1) 一个周期中0、1的个数相差不超过1个；
- (2) 一个周期段中，长度为 k 的游程占游程总数的 $1/2^k$ 这里假定至少有两个长为 k 的游程；
- (3) 周期自相关函数是二值函数。

凡满足这三条随机性假设的序列，被Golomb称为**伪随机序列**或者**伪噪声序列**。



2、m序列统计特性

1)、 m序列的 “0、1”信号的频次规律

性质1：r级m序列的一个周期中，1出现 2^{r-1} 个，
0出现 $2^{r-1} - 1$ 个。



2)、 m 序列的游程分布规律

若干个信号连续出现的现象称**游程**。对于序列 a ，称 a 中形如 $01\dots10$ 或 $10\dots01$ 的段为一个**1游程或0游程**，游程中所含1或0的个数称为该游程的长度，如 0110 为一个长为2的1游程， 101 为一个长为1的0游程。

性质2：将 r 级 m 序列的一个周期段首尾相接，其游程总数为 $N=2^{r-1}$ ；其中**没有**长度大于 r 的游程；有1个长度为 r 的1游程，没有长度为 r 的0游程；没有长度为 $r-1$ 的1游程，有1个长度为 $r-1$ 的0游程；有 2^{r-2-k} 个长度为 $k(1 \leq k \leq r-2)$ 的1游程，有 2^{r-2-k} 个长度为 $k(1 \leq k \leq r-2)$ 的0游程。



3)、m序列的自相关特性

若 $\underline{a} = (a_0 a_1 a_2 \dots)$ 是一个周期为 p 的 0、1 序列，定义 $\{0, 1\}$ 上的映射 η 为： $\eta(0) = 1, \eta(1) = -1$ ，定义序列 $\underline{a} = (a_0 a_1 a_2 \dots)$ 的自相关函数为

$$C(t) = \sum_{i=0}^{p-1} \eta(a_i) \eta(a_{i+t})$$

性质3： 若 $\underline{a} = (a_0 a_1 a_2 \dots)$ 是一个 r 级 m 序列，那么

$$C(t) = \sum_{i=0}^{2^r-2} \eta(a_i) \eta(a_{i+t}) = \begin{cases} 2^r - 1, & t = 0 \\ -1, & 0 < t < 2^r - 1 \end{cases}$$



Golomb随机性假设只是随机性的必要条件，一条随机序列应该满足Golomb随机性假设。但**满足上述标准的周期序列并不一定能满足我们对密钥流序列在安全性上的要求**，这种安全性是寓于随机性之中的。

例如：**m序列完全满足Golomb随机性假设，但m序列是高度可预测的。对于 r 级m序列，只要知道其连续 $2r$ 位，利用B-M算法就能以 $O(n^2)$ 的计算复杂度预测该序列的任一位的取值。**



线性移位寄存器的综合

已知**反馈多项式(或线性递推式)**及**初始状态**可获得所产生的**序列**。而**初始状态**总是可以假定的，故知道了**反馈多项式(或线性递推式)**也就知道了该**反馈多项式(或线性递推式)**所产生的序列。那么反过来，已知**序列**能否获得相应的**反馈多项式(或线性递推式)**呢？答案是肯定的。

(一)、解方程法

已知序列 a 是由 r 级线性移存器产生的，并且知道 a 的连续 $2r$ 位，可用解线性方程组的方法得到线性递推式。



例1、设序列 $a = (01111000\dots)$ 是由4级线性移存器所产生序列的连续8个信号，求该移存器的线性递推式。

解：设该4线移存器的线性递推式为：

$$a_n = c_1 a_{n-1} \oplus c_2 a_{n-2} \oplus c_3 a_{n-3} \oplus c_4 a_{n-4} \quad (n \geq 4)$$

由于知道周期序列的连续8个信号，不妨设为开头的8个信号，即

$$a_0 a_1 a_2 a_3 a_4 a_5 a_6 a_7 = 01111000$$

当 $n=4$ 时，由递归式可得： $a_4 = c_1 a_3 \oplus c_2 a_2 \oplus c_3 a_1 \oplus c_4 a_0$

即：

$$c_1 \oplus c_2 \oplus c_3 = 1 \quad (1)$$

同理可得：

$$c_1 \oplus c_2 \oplus c_3 \oplus c_4 = 0 \quad (2)$$

$$c_2 \oplus c_3 \oplus c_4 = 0 \quad (3)$$

$$c_3 \oplus c_4 = 0 \quad (4)$$

解方程组得

$$c_1 = 0, c_2 = 0, c_3 = 1, c_4 = 1$$

故所求移存器递推式为：

$$a_n = a_{n-3} \oplus a_{n-4} \quad (n \geq 4)$$



B-M迭代算法

根据密码学的需要，对线性反馈移位寄存器(LFSR)主要考虑下面两个问题：

(1) 如何利用级数尽可能短的LFSR产生周期大、随机性能良好的序列，即固定级数时，什么样的移存器序列周期最长。这是从密钥生成角度考虑，用最小的代价产生尽可能好的、参与密码变换的序列。

(2) 当已知一个长为 N 序列 a 时，如何构造一个级数尽可能小的LFSR来产生它。这是从密码分析角度来考虑，要想用线性方法重构密钥序列所必须付出的最小代价。这个问题可通过B-M算法来解决。



1)、概念简介

设 $\underline{a} = (a_0, a_1, \dots, a_{N-1})$ 是 F_2 上的长度为 N 的序列, 而

$f(x) = c_0 + c_1x + c_2x^2 + \dots + c_lx^l$ 是 F_2 上的多项式, $c_0=1$.

如果序列中的元素满足递推关系:

$$a_k = c_1a_{k-1} + c_2a_{k-2} + \dots + c_la_{k-l}, k = l, l+1, \dots, N-1 \quad (2)$$

则称 $\langle f(x), l \rangle$ 产生二元序列 $\underline{a}$ 。其中 $\langle f(x), l \rangle$ 表示以 $f(x)$ 为反馈多项式的 l 级线性移位寄存器。

如果 $f(x)$ 是一个能产生 $\underline{a}$ 并且级数最小的线性移位寄存器的反馈多项式, l 是该寄存器的级数, 则称 $\langle f(x), l \rangle$ 为 **序列 $\underline{a}$ 的线性综合解**。



序列的线性复杂度

一个无限序列 s 的线性复杂度 $L(s)$ 定义如下：

1. 若 s 是一个全零序列，则 $L(s)=0$
2. 若没有LFSR能生成 s ，则 $L(s)=\infty$
3. 否则， $L(s)$ 为能够产生 s 的最短LFSR

一个有限序列 s^n 的线性复杂度 $L(s^n)$ 定义为能够产生 s^n 的最短LFSR的长度。则：

1. $0 \leq L(s^n) \leq n$
2. $L(s^n)=0$ ，当且仅当 s^n 为长度为 n 的全零序列
3. $L(s^n)=n$ ，当且仅当 $s^n=00000 \dots 01$



线性移位寄存器的综合问题可表述为：给定一个 N 长二元序列 $\underline{a}$ ，如何求出产生这一序列的最小级数的线性移位寄存器，即最短的线性移存器？

几点说明：

1、**反馈多项式 $f(x)$ 的次数 $\leq l$** 。因为产生 $\underline{a}$ 且级数最小的线性移位寄存器可能是退化的，在这种情况下 $f(x)$ 的次数 $< l$ ；并且此时 $f(x)$ 中的 $c_l=0$ ，因此在反馈多项式 $f(x)$ 中 $c_0=1$ ，但不要求 $c_l=1$ 。

2、**规定：0级线性移位寄存器是以 $f(x)=1$ 为反馈多项式的线性移位寄存器，且 n 长($n=1, 2, \dots, N$)全零序列，仅由0级线性移位寄存器产生。**事实上，以 $f(x)=1$ 为反馈多项式的递归关系式是： $a_k=0, k=0, 1, \dots, n-1$ 。因此，这一规定是合理的。

3、给定一个 N 长二元序列 $\underline{a}$ ，求能产生 $\underline{a}$ 并且级数最小的线性移位寄存器，就是求 $\underline{a}$ 的**线性综合解**。**利用B-M算法可以有效地求出。**



2)、B-M算法要点

用归纳法求出一系列线性移位寄存器：

$$\langle f_n(x), l_n \rangle \quad \partial^0 f_n(x) \leq l_n, \quad n = 1, 2, \dots, N$$

每一个 $\langle f_n(x), l_n \rangle$ 都是产生序列 a 的前 n 项的最短线性移位寄存器，在 $\langle f_n(x), l_n \rangle$ 的基础上构造相应的 $\langle f_{n+1}(x), l_{n+1} \rangle$ ，使得 $\langle f_{n+1}(x), l_{n+1} \rangle$ 是产生给定序列前 $n+1$ 项的最短线性移位寄存器，则最后得到的 $\langle f_N(x), l_N \rangle$ 就是产生给定 N 长二元序列 a 的最短的线性移位寄存器。



3)、B-M算法

任意给定一个N长序列 $\underline{a} = (a_0, a_1, \dots, a_{N-1})$, 按n归纳定义
 $\langle f_n(x), l_n \rangle \quad n = 0, 1, 2, \dots, N-1$

1、取初始值: $f_0(x) = 1, \quad l_0 = 0$

2、设 $\langle f_0(x), l_0 \rangle, \langle f_1(x), l_1 \rangle, \dots, \langle f_n(x), l_n \rangle (0 \leq n < N)$

均已求得, 且 $l_0 \leq l_1 \leq \dots \leq l_n$

记: $f_n(x) = c_0^{(n)} + c_1^{(n)}x + \dots + c_{l_n}^{(n)}x^{l_n}, c_0^{(n)} = 1$, 再计算:

$$d_n = c_0^{(n)}a_n + c_1^{(n)}a_{n-1} + \dots + c_{l_n}^{(n)}a_{n-l_n}$$

称 d_n 为第n步差值。然后分两种情形讨论:



(i) 若 $d_n=0$, 则令:

$$f_{n+1}(x) = f_n(x), \quad l_{n+1} = l_n。$$

(ii) 若 $d_n=1$, 则需区分以下两种情形:

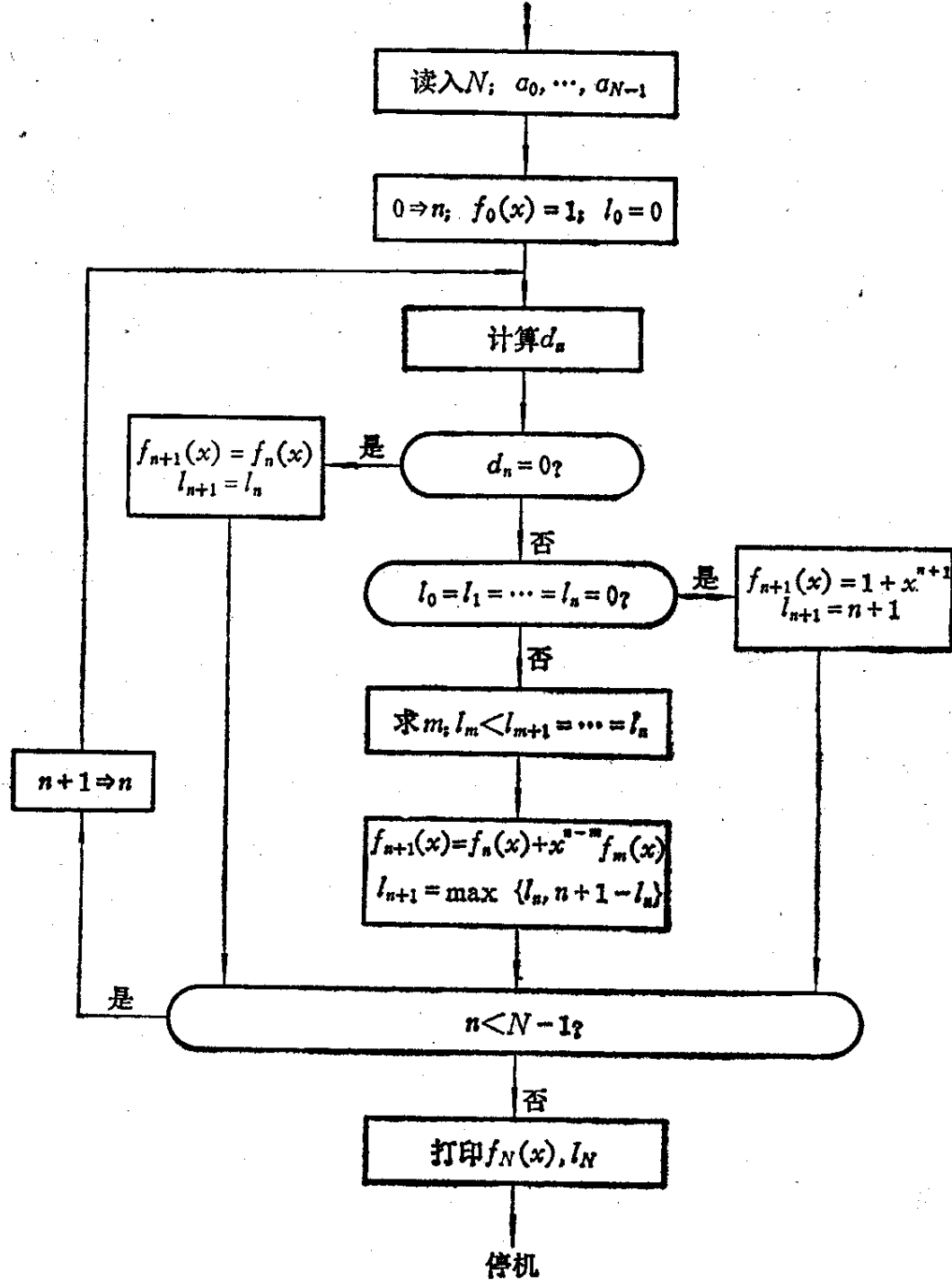
① 当: $l_0 = l_1 = L = l_n = 0$ 时,

$$\text{取: } f_{n+1}(x) = 1 + x^{n+1}, l_{n+1} = n + 1。$$

② 当有 m ($0 \leq m < n$), 使: $l_m < l_{m+1} = l_{m+2} = L = l_n$ 。

$$\text{则置: } f_{n+1}(x) = f_n(x) + x^{n-m} f_m(x), l_{n+1} = \max\{l_n, n + 1 - l_n\}$$

最后得到的 $\langle f_N(x), l_N \rangle$ 便是产生序列 a 的最短线性移位寄存器。该算法时间复杂度为 $O(N^2)$, 存储复杂度为 $O(N)$





4)、实例

例2、求产生周期为7的 m 序列一个周期：0011101的最短线性移位寄存器。

解：设 $a_0a_1a_2a_3a_4a_5a_6 = 0011101$, 首先取初值 $f_0(x)=1, l_0=0$, 则由 $a_0=0$ 得 $d_0=1 \cdot a_0=0$ 从而 $f_1(x)=1, l_1=0$; 同理由 $a_1=0$ 得 $d_1=1 \cdot a_1=0$ 从而 $f_2(x)=1, l_2=0$ 。

由 $a_2=1$ 得 $d_2=1 \cdot a_2=1$, 从而根据 $l_0 = l_1 = l_2 = 0$ 知
$$f_3(x) = 1 + x^{2+1} = 1 + x^3, l_3 = 3$$

第1步, 计算 d_3 : $d_3 = 1 \cdot a_3 + 0 \cdot a_2 + 0 \cdot a_1 + 1 \cdot a_0 = 1$
因为 $l_2 < l_3$, 故 $m=2$, 由此

$$f_4(x) = f_3(x) + x^{3-2} f_2(x) = 1 + x + x^3$$
$$l_4 = \max\{3, 3+1-3\} = \max\{3, 1\} = 3$$



第2步, 计算 d_4 : $d_4=1\cdot a_4 + 1\cdot a_3 + 0\cdot a_2 + 1\cdot a_1=0$, 从而

$$f_5(x) = f_4(x) = 1 + x + x^3$$

$$l_5 = l_4 = 3$$

第3步, 计算 d_5 : $d_5=1\cdot a_5 + 1\cdot a_4 + 0\cdot a_3 + 1\cdot a_2=0$, 从而

$$f_6(x) = f_5(x) = 1 + x + x^3$$

$$l_6 = l_5 = 3$$

第4步, 计算 d_6 : $d_6=1\cdot a_6 + 1\cdot a_5 + 0\cdot a_4 + 1\cdot a_3=0$, 从而

$$f_7(x) = f_6(x) = 1 + x + x^3$$

$$l_7 = l_6 = 3$$

这表明, $\langle 1 + x + x^3, 3 \rangle$ 即为产生所给序列一个周期的最短线性移位寄存器。



(五)、基于LFSR的伪随机序列生成器

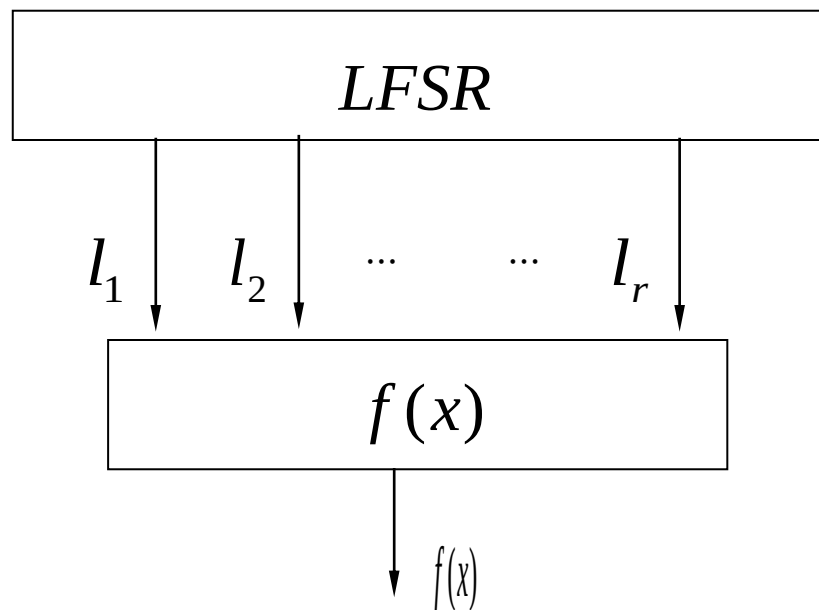
安全的密钥流应该满足三个条件：周期充分长、随机统计特性好、线性复杂度大。

----->LFSR+非线性化

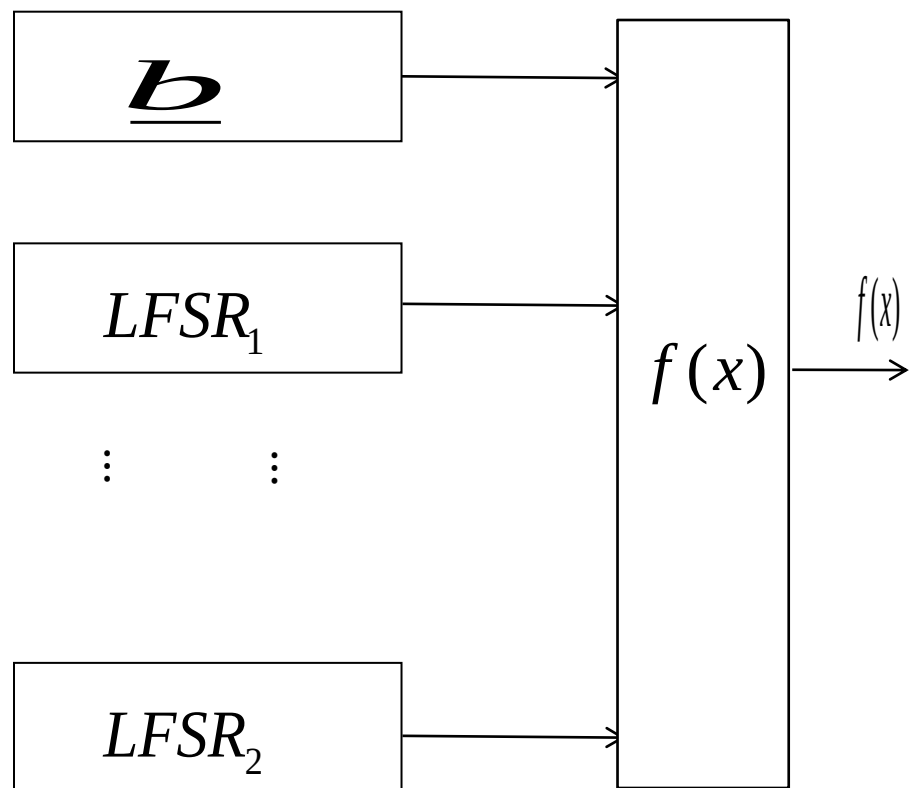
- 滤波生成器
- 组合生成器
- 钟控生成器
- Geffe序列生成器
- J-K触发器生成器



基于LFSR的密钥流生成器



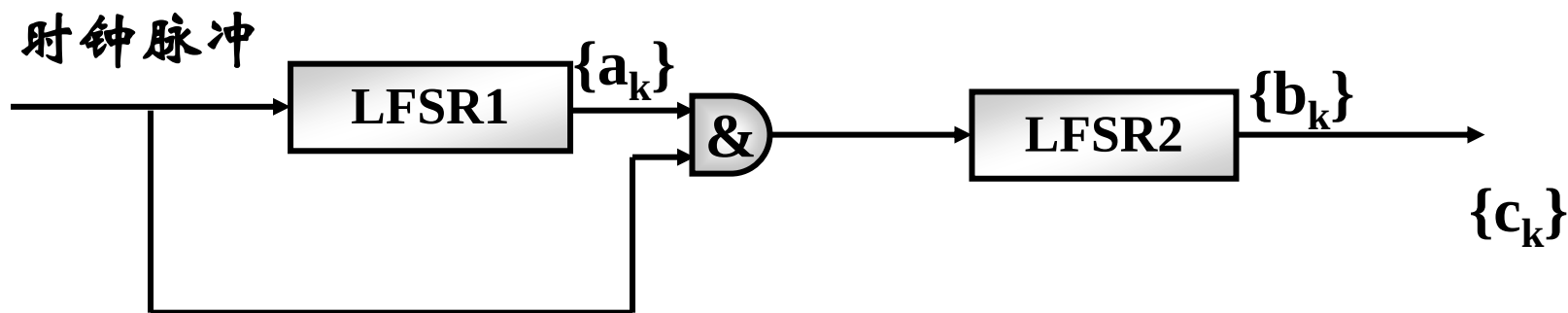
1: 非线性滤波生成器



2: 非线性组合生成器



□ 钟控生成器设计思想：用一个或多个移位寄存器来控制另一个或多个移位寄存器的时钟。
当LFSR1输出为1时，LFSR2移位输出，否则重复输出前一位。



eg:

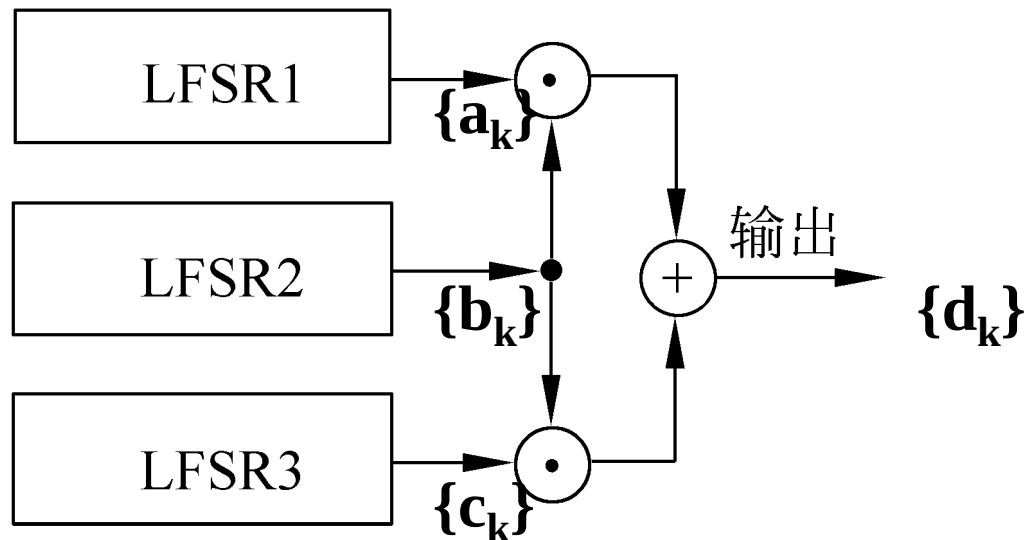
LFSR1的输出序列 $\{a_k\}$ 为：10101 10101

LFSR2的输出序列 $\{b_k\}$ 为：110 110 110

则整个钟控生成器的输出序列 $\{c_k\}$ 为：11110
11110, 周期为5.

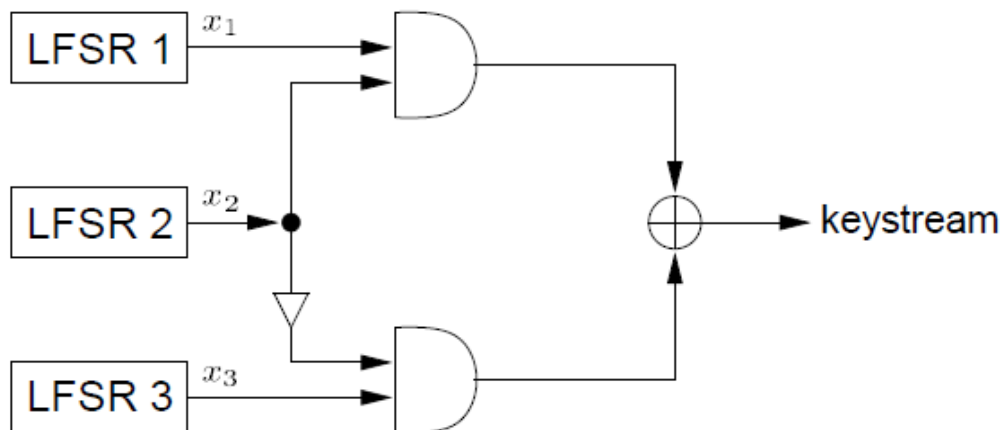


Geffe序列生成器由3个LFSR组成，其中LFSR2作为控制生成器使用。当LFSR2输出1时，LFSR2与LFSR1相连接；当LFSR2输出0时，LFSR2与LFSR3相连接。





Geffe generator



$$f(x_1, x_2, x_3) = x_1x_2 \oplus (1 + x_2)x_3 = x_1x_2 \oplus x_2x_3 \oplus x_3.$$

周期: $(2^{L_1} - 1) \cdot (2^{L_2} - 1) \cdot (2^{L_3} - 1)$

线性复杂度: $L = L_1L_2 + L_2L_3 + L_3$

相关性:

$$\begin{aligned} P(z(t) = x_1(t)) &= P(x_2(t) = 1) + P(x_2(t) = 0) \cdot P(x_3(t) = x_1(t)) \\ &= \frac{1}{2} + \frac{1}{2} \cdot \frac{1}{2} = \frac{3}{4}. \end{aligned}$$

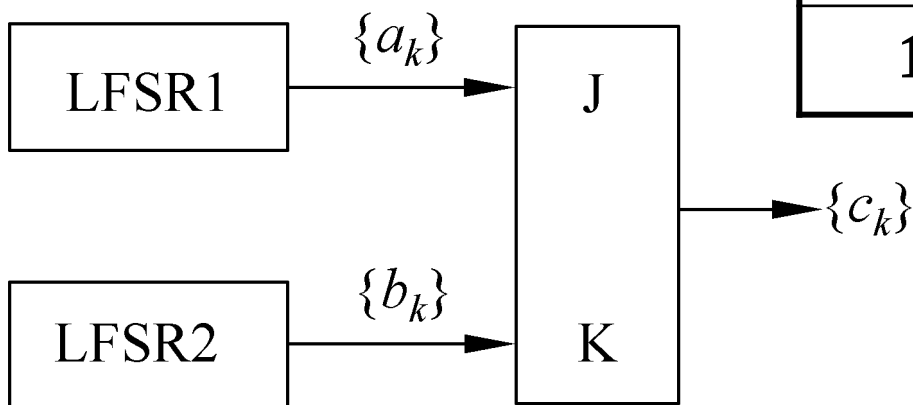


利用J-K触发器的非线性序列生成器

JK触发器的输出序列可以通过以下递推公式计算：

$$C_n = ((a_n + b_n + 1) \times c_{n-1} + a_n) \bmod 2$$

通常令 $c_0 = 0$



J-K触发器真值表

J	K	C_K
0	0	C_{K-1}
0	1	0
1	0	1
1	1	$\neg C_{K-1}$

取反



eg: 已知LFSR1生成周期为3的序列

$\{a_k\}=0,1,1,\dots$

LFSR2生成周期为4的序列

$\{b_k\}=1,0,0,1,\dots$

则生成器的输出序列 $\{c_k\}$ 是0110 1110 1110...

其周期为12。

J	K	C_K
0	0	C_{K-1}
0	1	0
1	0	1
1	1	$\neg C_{K-1}$



基于LFSR的密钥流生成器

- 驱动部分：LFSR
- 非线性组合部分 $f(x)$
 - 该变换可提高输入序列的线性复杂度
 - 该变换应保持输入序列良好的伪随机特性



eSTREAM获选算法

面向软件	设计理念	面向硬件	设计理念
HC-128	基于字的非线性更新函数的同步序列密码	Grain v1	基于NLFSR的同步序列密码
Rabbit	基于字的非线性更新函数的同步序列密码	MICKEY v2	钟控LFSR的同步序列密码
Salsa20/12	非线性状态转移函数的同步序列密码	Trivium	基于NLFSR的同步序列密码
SOSEMANUK	基于LFSR和FSM的同步序列密码		



(六) 实用序列密码

流密码的优点:

- 容易实现。
- 加密解密速度快。
- 无错误传播。

常用流密码算法

- RC4算法。
- A5算法--钟控生成器。



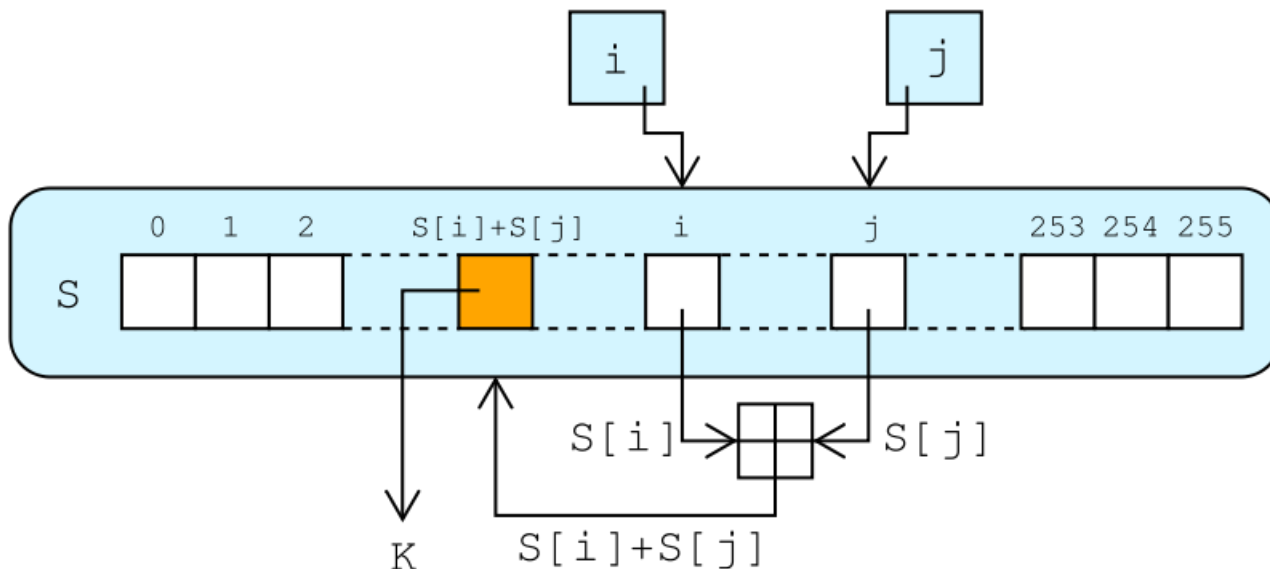
RC4算法

- **RC4算法的关键是能够高效产生不可预测的密钥序列。突出优点是在软件中很容易实现。**
- **RC4已应用于Windows、Lotus Notes和其他软件应用程序中，应用于安全套接字层SSL以保护因特网的信息流，还应用于无线系统以保护无线链路的安全。**



RC4算法

- RC4是典型的基于非线性数组变换的序列密码。
- 以一个足够大的数组为基础，对其进行非线性变换，产生非线性的密钥序列，一般把这个大数组叫做S盒。
- RC4算法可以生成总数为 $N=2^n$ 个元素的S盒。通常 n 取8。可以生成有256个元素的数组 s 。





RC4算法步骤

第一步：密钥调度算法KSA。

用于设置S盒内部状态($S[0], \dots, S[255]$)的随机排列。

step1:

//初始化数组S，将种子密钥K[L]的值付给数组T。

//使T与S等长。

for i=0 to 255 do

begin

$S[i]=i$; $T[i]=K[i \bmod L]$;

end;



step2:

//用T对S进行值的置换。

j=0;

for i=0 to 255 do

begin

j=(j+S[i]+T[i]) mod 256;

swap(S[i],S[j]);

end;



第二步：生成密钥。伪随机生成算法PRGA。

$i=0; j=0;$

重复下列步骤，直到获得足够长的密钥流。

$i=(i+1) \bmod 256;$

$j=(j+S[i]) \bmod 256;$

$\text{swap}(S[i], S[j]);$

$p=(S[i]+S[j]) \bmod 256;$

$k=S[p];$



示例

eg: 采用RC4算法对明文110101100加密。设
 $n=3$, $L=3$. 密钥种子 $K=\{5,6,7\}$ 。

解:

第一步: 密钥调度。

step1: 由于 $n=3$, 所以S盒的大小为 $2^3=8$ 。

初始化 $S=\{0,1,2,3,4,5,6,7\}$ 。

由于 $K=\{5,6,7\}$, 生成数组 $T=\{5,6,7,5,6,7,5,6\}$



step2:用T对S进行值的置换。

for i=0 to 7

$j=(j+S[i]+T[i]) \bmod 8;$

swap(S[i],S[j]);

T

5	6	7	5	6	7	5	6
---	---	---	---	---	---	---	---

$i=0, j=0$

S

0	1	2	3	4	5	6	7
---	---	---	---	---	---	---	---

(1) $i=0, j=5$

S

5	1	2	3	4	0	6	7
---	---	---	---	---	---	---	---

(2) $i=1, j=4$

S

5	4	2	3	1	0	6	7
---	---	---	---	---	---	---	---

(3) $i=2, j=5$

S

5	4	0	3	1	2	6	7
---	---	---	---	---	---	---	---



step2:用T对S进行值的置换。

for i=0 to 7

$j = (j + S[i] + T[i]) \bmod 8;$

swap(S[i],S[j]);

T

5	6	7	5	6	7	5	6
---	---	---	---	---	---	---	---

(4)i=3,j=5

S

5	4	0	2	1	3	6	7
---	---	---	---	---	---	---	---

(5)i=4,j=4

S

5	4	0	2	1	3	6	7
---	---	---	---	---	---	---	---

(6)i=5,j=6

S

5	4	0	2	1	6	3	7
---	---	---	---	---	---	---	---

(8)i=7,j=3

S

5	4	0	7	1	6	3	2
---	---	---	---	---	---	---	---

(7)i=6,j=6

S

5	4	0	2	1	6	3	7
---	---	---	---	---	---	---	---



第二步：生成密钥。

$i=0; j=0;$

重复下列步骤，直到获得足够长的密钥流。

$i=(i+1) \bmod 8;$

$j=(j+S[i]) \bmod 8;$

$\text{swap}(S[i], S[j]);$

$p=(S[i]+S[j]) \bmod 8;$

$k=S[p];$

明文:110101100

S

5	4	0	7	1	6	3	2
---	---	---	---	---	---	---	---

$i=0, j=0$

(1) $i=1, j=4$

S

5	1	0	7	4	6	3	2
---	---	---	---	---	---	---	---

$p=5, k=6=(110)$

(2) $i=2, j=4$

S

5	1	4	7	0	6	3	2
---	---	---	---	---	---	---	---

$p=4, k=0=(000)$



第二步：生成密钥。

$i=0; j=0;$

重复下列步骤，直到获得足够长的密钥流。

$i=(i+1) \bmod 8;$

$j=(j+S[i]) \bmod 8;$

$\text{swap}(S[i], S[j]);$

$p=(S[i]+S[j]) \bmod 8;$

$k=S[p];$

明文:110101100

$(3)i=3, j=3$

S

5	1	4	7	0	6	3	2
---	---	---	---	---	---	---	---

$p=6, k=3=(011)$

所以密文C=

$(110101100) \oplus (110000011)$
 $=000101111$



A5/1序列密码算法

- 用于手机到基站之间的通信加密
- 每次通话内容按每帧228比特加密
- 每帧采用不同的会话密钥（22比特,帧序号）



A5/1 LFSRs

Consists of 3 LFSRs of different lengths

□ 19 bits

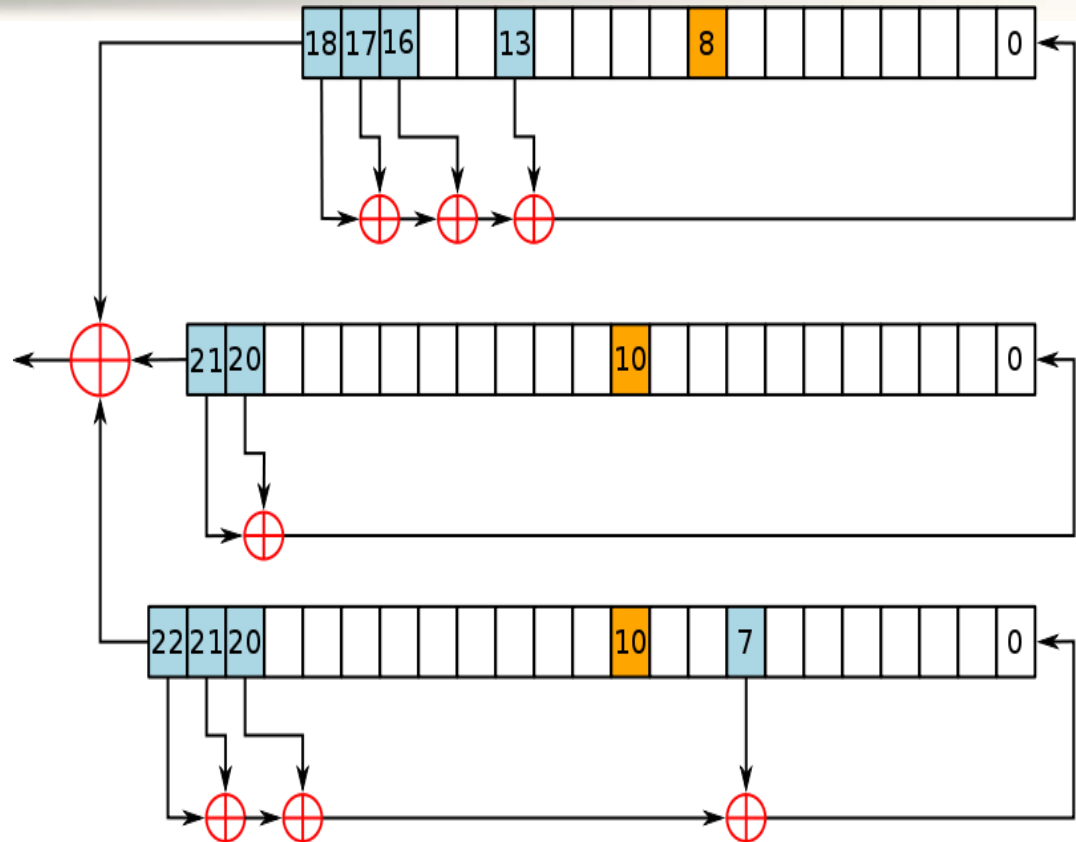
- $x^{18} + x^{17} + x^{16} + x^{13} + 1$
 - clock bit 8
- tapped bits: 13, 16, 17, 18

□ 22 bits

- $x^{21} + x^{20} + 1$
 - clock bit 10
- tapped bits 20, 21

□ 23 bits

- $x^{22} + x^{21} + x^{20} + x^7 + 1$
 - clock bit 10
- tapped bits 7, 20, 21, 22



64比特密钥+22比特帧数；钟控： Majority原则， 每帧228比特

钟控机制是：若某一时刻钟控单元输入的三个值中有两个或三个相同，则对应的寄存器下一刻被驱动，剩下的停走。



A5/1工作流程

- 状态初始化阶段
 - 3个LFSR全设为0
 - 3个LFSR规则动作64步，每次动作一步，反馈内容与密钥第i比特XOR
 - 3个LFSR规则动作22步，每次动作一步，反馈内容与帧序号第i比特XOR
- 密钥流（乱数）输出
 - 3个LFSR以钟控方式连续动作100次，不输出乱数
 - 3个LFSR以钟控方式连续动作114次，输出114比特乱数，用于手机到基站的114比特数据的加密。
 - 3个LFSR以钟控方式连续动作100次，不输出乱数
 - 3个LFSR以钟控方式连续动作114次，输出114比特乱数，用于基站到手机的114比特数据的加密。



Trivium

- ❑ 基于非线性反馈移位寄存器
- ❑ eSTREAM计划的胜出算法，已被纳入ISO/IEC标准
- ❑ 密钥80比特，IV 80比特



Trivium

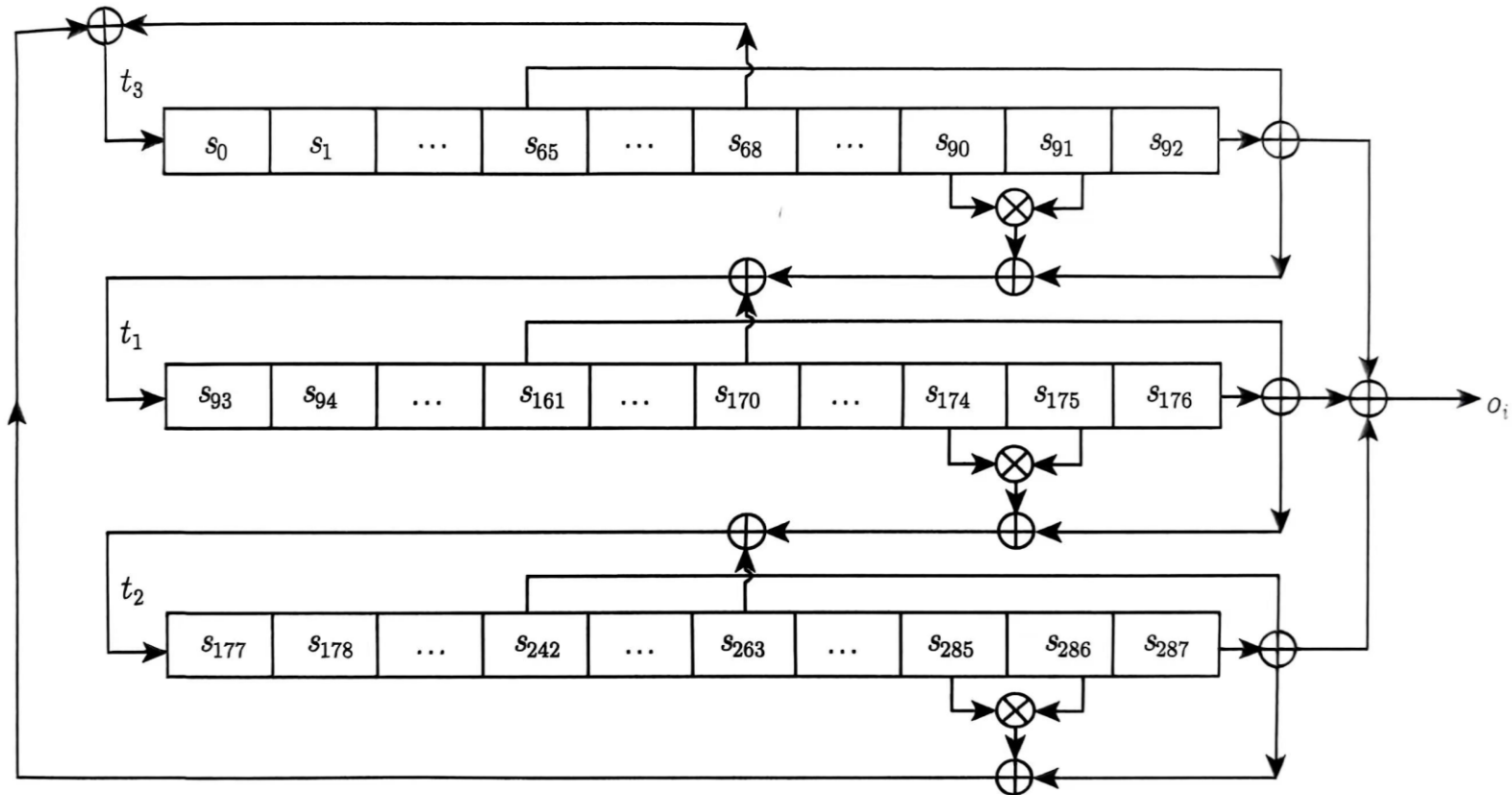


图 5.10 流密码标准 Trivium 算法的密钥流生成过程



Trimium

设TRIVIUM内部状态为 $S_1, \dots, S_{288}$, 初始化密钥为 $K_1 \dots K_{80}$, 初始化向量为 $IV_1 \dots IV_{80}$, 则初始化过程为

$$(S_0, S_1, \dots, S_{92}) = (K_0, K_1, \dots, K_{79}, 0, 0, \dots, 0)$$

$$(S_{93}, S_{94}, \dots, S_{176}) = (IV_0, IV_2, \dots, IV_{79}, 0, 0, \dots, 0)$$

$$(S_{177}, S_{178}, \dots, S_{285}, S_{286}, S_{287}) = (0, 0, \dots, 0, 1, 1, 1)$$

For iter = 1 to $4 * 288$ do

$$T_1 = S_{65} + S_{90} * S_{91} + S_{92} + S_{170}$$

$$T_2 = S_{161} + S_{174} * S_{175} + S_{176} + S_{263}$$

$$T_3 = S_{242} + S_{285} * S_{286} + S_{287} + S_{68}$$

$$(S_0, S_1, \dots, S_{92}) = (T_3, S_0, \dots, S_{91})$$

$$(S_{93}, S_{94}, \dots, S_{176}) = (T_1, S_{93}, \dots, S_{175})$$

$$(S_{177}, S_{178}, \dots, S_{287}) = (T_2, S_{177}, \dots, S_{286})$$

End For

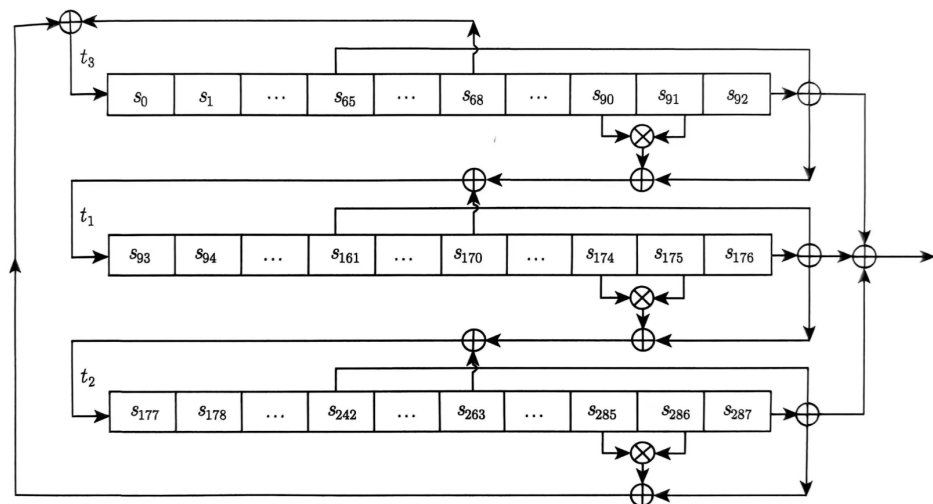


图 5.10 流密码标准 Trivium 算法的密钥流生成过程



Trivium

2 生成密钥流

设需生成N个密钥位，第i个输出记为 Z_i ，有

For iter = 1 to N do

$$T1 = S_{65} + S_{92}$$

$$T2 = S_{161} + S_{176}$$

$$T3 = S_{242} + S_{287}$$

$$Z_i = T1 + T2 + T3$$

$$T1 = S_{65} + S_{90} * S_{91} + S_{92} + S_{170}$$

$$T2 = S_{161} + S_{174} * S_{175} + S_{176} + S_{263}$$

$$T3 = S_{242} + S_{285} * S_{286} + S_{287} + S_{68}$$

$$(S_0, S_1, \dots, S_{92}) = (T3, S_0, \dots, S_{91})$$

$$(S_{93}, S_{94}, \dots, S_{176}) = (T1, S_{93}, \dots, S_{175})$$

$$(S_{177}, S_{178}, \dots, S_{287}) = (T2, S_{177}, \dots, S_{286})$$

End For

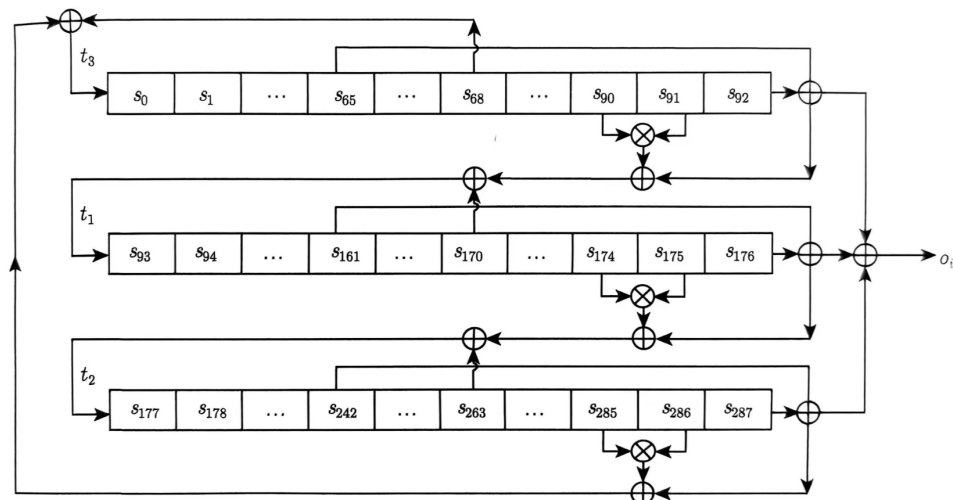


图 5.10 流密码标准 Trivium 算法的密钥流生成过程



谢 谢！